BLOQUO × PESSIMISTIC

SECURITY ANALYSIS

by Pessimistic

This report is public

September 29, 2025

Abstract 2
Disclaimer 2
Summary 2
General recommendations 2
Project overview 3
Project description 3
Codebase update 3
Audit process 4
Manual analysis 5
Critical issues 5
Medium severity issues 6
M01. Withdrawal request can be front-run (addressed) 6
M02. Rounding allows zero-asset locks (fixed) 6
Low severity issues 7
L01. Inline comment inside condition reduces readability (fixed) 7
L02. Zero-share withdrawal request (fixed) 7
L03. Partial withdraw with ceiling rounding (fixed) 7
L04. TODO comments left in production contracts (fixed) 8
L05. Redundant require wastes gas (fixed) 8
L06. Use immutable (fixed) 8
L08. User experience (commented) 8
Notes 9
N01. Fee on transfer and rebase tokens are not supported 9
N02. Project roles 9

ABSTRACT

This report evaluates the security of the smart contracts in the Bloquo project. The objective was to identify and document security vulnerabilities within the platform's smart contracts.

DISCLAIMER

This audit makes no warranties or guarantees as to the security of the code. A single audit is insufficient to ensure complete security. We strongly recommend conducting multiple independent audits and implementing a public bug bounty program to enhance smart contract security. Additionally, this security audit should not be construed as investment advice.

SUMMARY

This report assesses the security of the Bloquo smart contracts. The audit process is detailed in the following section.

The audit identified two medium severity issues: Withdrawal request can be front-run, Rounding allows zero-asset locks. Also, several low severity issues were found.

After the initial audit, the codebase was updated. All the issues were fixed or commented on. One low-severity issue (L07) was identified as false positive and removed.

The overall code quality is good.

GENERAL RECOMMENDATIONS

We do not have any further recommendations.

PROJECT OVERVIEW

Project description

For this audit, we reviewed the Bloquo project from a private GitHub repository (commit f30b4fdeafb1ba188e61d4275def7dde66e247b2).

The scope of the audit included everything.

The provided documentation included the following link.

The list of networks for deployment:

- Ethereum.

All 8 tests pass successfully. The code coverage is 94.33%.

The total lines of code (LOC) of audited sources is 708.

Codebase update

After the initial audit, the codebase was updated. For the recheck, we were provided with the new commit 536267d838a1a06f2120e33f1f77f8584bf5f1ec.

This update included fixes or comments for all issues. The number of tests increased. All 11 tests passed. The code coverage is 94.12%. Moreover, the developers added CI to run tests in the repository.

AUDIT PROCESS

The audit was conducted from September 16, 2025, to September 23, 2025.

We began by reviewing the provided materials, then met with the development team for a project overview.

We performed a comprehensive manual review of all contracts within the audit scope, verifying the logic and security properties, including:

- Tokens cannot be stolen from vaults;
- Inflation attack is not possible;
- Possilbe issues with roundings;
- Standard Solidity issues.

We analyzed the project with the following automated tools:

- AI scanner AuditAgent;
- AI scanner Savant Chat;
- Static analyzer Slither;
- Our plugin Slitherin with an extended set of rules;
- Semgrep rules for smart contracts.

We executed the test suite and calculated code coverage.

All identified issues — whether detected manually or through automated tools — were compiled into the private report.

After the initial audit, we discussed the results with the developers. On Septemper 25 the developers provided us with an updated version of the code. In this update, most of the issues were resolved, and the remaining ones were addressed with comments. One low-severity issue L07 was removed as false positive.

We reviewed the updated codebase and confirmed that previously reported issues had been resolved.

Finally, we updated the report.

MANUAL ANALYSIS

All contracts were manually inspected to validate their logic and security. Additionally, automated analysis results were manually verified. Confirmed issues are documented below.

Critical issues

Critical issues pose severe risks to project security, potentially leading to fund loss or other catastrophic failures. Contracts should not be deployed until such issues are resolved.

No critical issues were identified during the audit.

Medium severity issues

Medium-severity issues may impact the project's current functionality. This category includes bugs, potential revenue loss, operational inefficiencies, and risks related to improper system management. We strongly recommend addressing these issues.

M01. Withdrawal request can be front-run (addressed)

In **BloquoInvestmentVault.sol**, `request` function locks a user's claimable `assets` at the current exchange rate. If a loss is realized via `divest` or `depositInvestmentReturns`, a user can front-run by calling `request` to fix the pre-loss rate. After the loss, their locked `_requests[addr].assets` remains unchanged, pushing the loss onto non-requesting holders and enabling unfair withdrawals.

> `Comment from the developers:`
>
> Bloquo is using private mempools/MEV-protected nodes for posting `invest()`/`divest()` transactions.

M02. Rounding allows zero-asset locks (fixed)

In **BloquoInvestmentVault.sol**, `request` function computes `assets` via `convertToAssets(shares, Math.Rounding.Floor)` and permits `assets == 0` for non-zero `shares_`. It then increments `sharesLocked` by `shares` but `_assetsLocked` by zero, skewing the unlocked pricing basis `(totalAssets() - _assetsLocked) / (totalSupply() - _sharesLocked)`. This temporarily inflates the share price so new deposits mint fewer shares, shifting value to existing holders (including the attacker). After the victim deposits, the attacker redeems the zero-asset request, restoring counters but keeping the extracted value.

> The issue has been fixed and is not present in the latest version of the code.

Low severity issues

Low severity issues do not immediately affect project operations but may introduce risks in future iterations. We recommend resolving these issues or providing justification for the chosen implementation.

L01. Inline comment inside condition reduces readability (fixed)

At line 413 in **BloquoInvestmentVault.sol**, `depositInvestmentReturns` places a block comment inside the `if` condition: `/* amountInvested_ == 0 || */ amountInvested_ > assetsInvestedDirectly`. This makes the condition harder to read and maintain.

> The issue has been fixed and is not present in the latest version of the code.

L02. Zero-share withdrawal request (fixed)

In **BloquoInvestmentVault.sol**, `request` function accepts `shares_ == 0`, setting `_requests[account].requested = true` while locking zero shares/assets. Since `request` later requires `requested == false` and `withdraw`/`redeem` (burn path) require positive locked shares, the flag cannot be cleared and the account cannot progress. This causes a per-account denial of service on withdrawals. The user can still transfer shares and request from another address, but the original address remains stuck.

> The issue has been fixed and is not present in the latest version of the code.

L03. Partial withdraw with ceiling rounding (fixed)

In **BloquoInvestmentVault.sol**, `withdraw` function computes shares to burn using ceiling rounding. When `assets_per_share > 1` and a user withdraws less than the full requested assets, this can burn all locked shares while leaving a positive `_requests[account].assets`. The subsequent burn clears the `requested` flag, but the remaining assets stay recorded in the request and continue to count in `_assetsLocked` with no path to release them, permanently locking funds and reducing vault liquidity.

> The issue has been fixed and is not present in the latest version of the code.

L04. TODO comments left in production contracts (fixed)

The codebase contains TODO comments. Leaving TODOs in production code indicates that some aspects of the implementation may not be fully thought through or completed. We recommend resolving or removing these before deployment. Found at lines 13–16, 252–253, 285 of the **BloquoInvestmentVault** contract and at lines 341, 358, 375, 392, 409 of the **BloquoOpportunityVault** contract.

> The issue has been fixed and is not present in the latest version of the code.

L05. Redundant require wastes gas (fixed)

At lines 35–36 of the **AbstractOwnableRescuable** contract, `rescue` function performs two checks for the same condition. The second check `require(amount_ <= myBalance(token_) - lockedAssets(token_))` already ensures the rescue does not exceed available (unlocked) assets, so the first `require(myBalance(token_) >= lockedAssets(token_))` becomes redundant and only increases gas usage. Consider removing the first require.

> The issue has been fixed and is not present in the latest version of the code.

L06. Use immutable (fixed)

At line 168 of the **BloquoInvestmentVault** contract, `withdrawalWaitPeriodChangesAllowed` is only assigned in the `constructor` and never modified afterward. Consider marking it `immutable` to save gas.

> The issue has been fixed and is not present in the latest version of the code.

L08. User experience (commented)

Enable users to cancel or partially reduce a pending withdrawal request in the **BloquoInvestmentVault** contract. Because withdrawals may require a waiting period, users can change their intent after submitting a request. Allowing cancellation or partial cancellation lets them keep some or all funds in the vault without waiting the full period and then re-depositing.

> `Comment from the developers:`
> Noted, but functions as intended. This approach avoids issues with exchange rate manipulation and excessive computational complexity. Retail clients desiring greater liquidity of their investments will presumably swap the vault tokens through liquidity providers.

Notes

N01. Fee on transfer and rebase tokens are not supported

Fee on transfer and rebase tokens will break calculations, therefore they are not supported.

> According to the developers, only stablecoins tokens will be used as asset.

N02. Project roles

In the current implementation, the system depends heavily on the owner role. Thus, there are scenarios that can lead to undesirable consequences for the project and its users, e.g., if owner's private keys become compromised. In the project, the owner can:

- Recover stuck tokens;
- Set withdrawal wait period;
- Start investments and move funds;
- Record investment returns;
- Cancel or close opportunities;
- Distribute rewards;
- Pause or resume the contract;
- Adjust funding target and overfunding cap;
- Set funding deadline;
- Set expected reward time.

We recommend designing contracts in a trustless manner or implementing proper key management, e.g., setting up a multisig.

> According to the developers, owner role will be managed by multisig.

This analysis was performed by Pessimistic:

Yhtyyar Sahatov, Security Engineer
Daria Korepanova, Senior Security Engineer
Irina Vikhareva, Project Manager
Alexander Seleznev, CEO

September 29, 2025